

Assembly Code Generation for Lab Topics

<https://gemini.google.com/app/0efebdb9e9b74202>

User prompt: `.model small .stack 100h .code main proc mov ah,1 ;for 1 ah become input function int 21h ;stored in al mov bl,al ;store input to bl low part mov dl,' ' ;move space to dl output mov ah,2 int 21h ;prtn space mov ah,1 ;for 1 ah become input function int 21h ;stored in al mov bh,al ;store input to bh high part mov dl,' ' ;move space to dl output mov ah,2 int 21h ;prtn space mov dl,bl ;move value to print mov ah,2 ;for 2 ah become output function int 21h ; print mov dl,' ' ;move space to dl output mov ah,2 int 21h ;prtn space mov dl,bh ;move value to print mov ah,2 ;for 2 ah become output function int 21h ; print mov ah,4ch ;terminate program int 21h ;exicute it main endp end main, ,,,give this type of code for all these topics, don't miss any topic, give 30 code of each topic`

Response: I see you are working through the Microprocessor Assembly lab syllabus shown in the image "1000081978.jpg".

To be completely candid, generating 30 complete, distinct code files for *each* of the 6 topics (180 full assembly programs) in a single message goes well beyond the output limits of this chat window. However, I want to make sure you have exactly what you need to succeed in your lab.

Below, I have written a foundational, heavily commented code template for **every single topic** listed in your syllabus. These perfectly match the `.model small` and DOS interrupt structure you provided. You can use these as your core templates to build your 30 variations.

1. Addition (ADD)

This program takes two numbers as input (e.g., '2' and '3'), adds them, and prints the result. (Note: To avoid ASCII characters getting messy, input numbers that sum to 9 or less).

```
.model small
.stack 100h
.code
main proc
    ; Input first number
    mov ah, 1
    int 21h
    mov bl, al
    sub bl, 48    ; Convert ASCII to actual decimal value

    ; Input second number
    mov ah, 1
    int 21h
    sub al, 48    ; Convert ASCII to actual decimal value

    ; Add the numbers
    add bl, al    ; BL = BL + AL
    add bl, 48    ; Convert sum back to ASCII for printing

    ; Print a newline for neatness
    mov ah, 2
    mov dl, 10    ; Line feed
    int 21h
    mov dl, 13    ; Carriage return
    int 21h

    ; Print the result
    mov dl, bl
    mov ah, 2
    int 21h

    ; Terminate
    mov ah, 4ch
    int 21h
main endp
end main
```

2. Subtraction (SUB)

This takes two numbers and subtracts the second from the first. (*Input a larger number first, then a smaller one*).

```
.model small
.stack 100h
.code
main proc
    ; Input first number
    mov ah, 1
    int 21h
    mov bl, al

    ; Input second number
    mov ah, 1
    int 21h
    mov cl, al

    ; Subtract
    sub bl, cl    ; BL = BL - CL (Subtracts ASCII values directly, which works for single digits)
    add bl, 48    ; Readjust ASCII offset

    ; Print space
    mov ah, 2
    mov dl, ' '
    int 21h

    ; Print the result
    mov dl, bl
    mov ah, 2
    int 21h

    ; Terminate
    mov ah, 4ch
    int 21h
main endp
end main
```

3. Multiplication (MUL)

Multiplication works a bit differently because it automatically uses the AL register and outputs to AX. This multiplies 2 and 3 and prints '6'.

```
.model small
.stack 100h
.code
main proc
    mov al, 3    ; First multiplier in AL
    mov bl, 2    ; Second multiplier in BL

    mul bl       ; Multiplies AL by BL. Result is stored in AX. (Since result is small, it's in AL)

    ; Print the result
    mov dl, al
    add dl, 48    ; Convert decimal 6 back to ASCII '6'

    mov ah, 2
    int 21h

    ; Terminate
    mov ah, 4ch
    int 21h
main endp
end main
```

4. Division (DIV)

Similar to multiplication, division uses AX. This divides 8 by 2 and prints the quotient '4'.

```
.model small
.stack 100h
.code
main proc
    mov ax, 8      ; Dividend in AX
    mov bl, 2      ; Divisor in BL

    div bl         ; Divides AX by BL. Quotient goes to AL, Remainder goes to AH.

    ; Print the quotient
    mov dl, al
    add dl, 48     ; Convert decimal 4 back to ASCII '4'

    mov ah, 2
    int 21h

    ; Terminate
    mov ah, 4ch
    int 21h
main endp
end main
```

5. Comparison & Branching (CMP, JE, JNE, JMP)

This takes two inputs. If they are the same (Jump if Equal), it prints 'Y'. If they are different (Jump if Not Equal), it jumps to a different block and prints 'N'.

```
.model small
.stack 100h
.code
main proc
    ; Input 1
    mov ah, 1
    int 21h
    mov bl, al

    ; Input 2
    mov ah, 1
    int 21h

    ; Compare AL (Input 2) with BL (Input 1)
    cmp al, bl
    je ARE_EQUAL ; Jump to ARE_EQUAL label if they match

ARE_NOT_EQUAL:
    ; This block runs if they don't match
    mov dl, 'N'
    jmp PRINT    ; Unconditional jump to skip the EQUAL block

ARE_EQUAL:
    ; This block runs if they DO match
    mov dl, 'Y'

PRINT:
    ; Print whatever is in DL
    mov ah, 2
    int 21h

    ; Terminate
    mov ah, 4ch
    int 21h
```

```
main endp
end main
```

6. Loops (LOOP)

This program uses the CX register as a counter to print the numbers 1 through 5 in a sequence.

```
.model small
.stack 100h
.code
main proc
    mov cx, 5      ; Set loop counter to 5
    mov dl, '1'    ; Start with ASCII '1'

PRINT_LOOP:
    ; Print the character in DL
    mov ah, 2
    int 21h

    ; Print a space
    mov bl, dl     ; save current number
    mov dl, ' '
    int 21h
    mov dl, bl     ; restore current number

    inc dl         ; Increment to the next number (e.g., '1' becomes '2')

    loop PRINT_LOOP ; Decrements CX by 1. If CX is not 0, jumps back to PRINT_LOOP

    ; Terminate
    mov ah, 4ch
    int 21h
main endp
end main
```

User prompt: You don't give each topic code? Give all the code

Response: I understand it can be frustrating if it feels like you didn't get what you needed. To be completely transparent, generating 180 distinct assembly programs (30 variations for all 6 topics) in a single response exceeds the text limits of this system—the response would simply cut off halfway through.

However, looking closely at your syllabus in "1000081978_2.jpg", I see that there are specific sub-topics I didn't separate out last time (like **JC** and **Nested Loops**).

To give you exactly what you need for your lab, here is a complete, distinct `.model small` program for **every single specific instruction** listed in your syllabus image. You can copy, run, and modify these to fulfill your lab requirements.

1. Addition (ADD)

This program adds two hardcoded numbers (3 and 4) and prints the result (7).

```
.model small
.stack 100h
.code
main proc
    mov bl, 3      ; First number
    mov al, 4      ; Second number

    add bl, al     ; BL = BL + AL (3 + 4 = 7)

    add bl, 48     ; Convert to ASCII character '7' for printing

    mov dl, bl
```

```

    mov ah, 2      ; DOS output function
    int 21h

    mov ah, 4ch    ; Terminate program
    int 21h
main endp
end main

```

2. Subtraction (SUB)

This program subtracts 2 from 7 and prints the result (5).

```

.model small
.stack 100h
.code
main proc
    mov bl, 7      ; Number to subtract from
    mov al, 2      ; Amount to subtract

    sub bl, al     ; BL = BL - AL (7 - 2 = 5)

    add bl, 48     ; Convert to ASCII character '5'

    mov dl, bl
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

3. Multiplication (MUL)

Multiplies 2 and 4. Multiplication uses the AL register and outputs to AX.

```

.model small
.stack 100h
.code
main proc
    mov al, 2      ; First multiplier MUST be in AL
    mov bl, 4      ; Second multiplier

    mul bl         ; Multiplies AL * BL. Result goes to AX.

    mov dl, al     ; Since the result (8) is small, it fits in AL
    add dl, 48     ; Convert to ASCII

    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

4. Division (DIV)

Divides 9 by 3. Division uses AX for the dividend.

```

.model small
.stack 100h
.code
main proc

```

```

    mov ax, 9      ; Dividend MUST be in AX
    mov bl, 3      ; Divisor

    div bl         ; AX / BL. Quotient goes to AL, Remainder to AH.

    mov dl, al     ; Move quotient to DL for printing
    add dl, 48     ; Convert to ASCII

    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

5. Conditional Branching (JE, JNE, JMP)

This program compares two values. If they are equal, it jumps to a specific block using JE. It uses JMP to skip over code so both blocks don't run.

```

.model small
.stack 100h
.code
main proc
    mov al, 5
    mov bl, 5

    cmp al, bl     ; Compare AL and BL
    je EQUAL       ; Jump if Equal (JE)

    ; If NOT equal (JNE logic naturally falls here)
    mov dl, 'N'
    jmp PRINT      ; Unconditional Jump (JMP) to skip the equal part

EQUAL:
    mov dl, 'Y'    ; This runs if JE triggers

PRINT:
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

6. Branching on Carry (JC)

When an operation overflows an 8-bit register (exceeds 255), the Carry Flag is set. JC (Jump if Carry) detects this.

```

.model small
.stack 100h
.code
main proc
    mov al, 255    ; Maximum 8-bit value
    add al, 1      ; This causes an overflow/carry

    jc HAS_CARRY   ; Jump if Carry flag is set (JC)

    ; If no carry
    mov dl, 'N'
    jmp EXIT

```

```

HAS_CARRY:
    mov dl, 'C'    ; 'C' for Carry

EXIT:
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

7. Single Loop (LOOP)

The LOOP instruction automatically uses the CX register as a counter and decrements it until it hits 0.

```

.model small
.stack 100h
.code
main proc
    mov cx, 5      ; Set loop counter to 5
    mov dl, 'A'    ; Character to print

MY_LOOP:
    mov ah, 2
    int 21h        ; Print character

    inc dl         ; Move to next letter (A -> B -> C...)

    loop MY_LOOP   ; Decrements CX, jumps back if CX != 0

    mov ah, 4ch
    int 21h
main endp
end main

```

8. Nested Loops

To loop inside another loop, you must save the outer loop's CX counter to the stack (push cx) so the inner loop doesn't overwrite it, and restore it later (pop cx). This prints a 3x3 grid of stars.

```

.model small
.stack 100h
.code
main proc
    mov cx, 3      ; Outer loop counter (3 rows)

OUTER:
    push cx        ; **Save** outer loop counter to stack
    mov cx, 3      ; Inner loop counter (3 stars per row)

INNER:
    mov dl, '*'    ; Print a star
    mov ah, 2
    int 21h
    loop INNER     ; Loop inner 3 times

    ; Print Newline (Carriage Return + Line Feed)
    mov dl, 10
    mov ah, 2
    int 21h
    mov dl, 13
    int 21h

```

```

    pop cx          ; **Restore** outer loop counter from stack
    loop OUTER      ; Loop outer 3 times

    mov ah, 4ch
    int 21h
main endp
end main

```

User prompt: Give jump related 10 code of each topic

Response: I completely understand why you want more variations to study from. Having multiple examples of how branching works is the best way to prepare for an assembly lab.

Because generating 40 full assembly programs (10 for each of the 4 jump instructions) in a single message exceeds the text limit I can output at once, I have curated **10 distinct, highly useful code variations** that thoroughly cover JMP, JE, JNE, and JC. These examples demonstrate the exact patterns you will likely be asked to write in your lab exam.

Here are 10 complete, runnable variations focusing entirely on branching logic:

1. Unconditional Jump (JMP) - Skipping Code

This program demonstrates how JMP forces the program to skip over instructions. It jumps right over the code that prints 'A' and prints 'B' instead.

```

.model small
.stack 100h
.code
main proc
    jmp SKIP_A      ; Unconditionally jump to SKIP_A label

    ; This code gets skipped completely
    mov dl, 'A'
    mov ah, 2
    int 21h

SKIP_A:
    mov dl, 'B'     ; Program resumes here
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

2. Jump if Equal (JE) - Hardcoded Number Check

This compares two numbers in registers. If they match, it prints 'E' (Equal).

```

.model small
.stack 100h
.code
main proc
    mov al, 8
    mov bl, 8

    cmp al, bl      ; Compare the two registers
    je IS_EQUAL    ; Jump to IS_EQUAL if AL == BL

    jmp END_PROG    ; If not equal, jump to end (skip printing)

IS_EQUAL:
    mov dl, 'E'

```



```

    mov ah, 2
    int 21h

END_PROG:
    mov ah, 4ch
    int 21h
main endp
end main

```

3. Jump if Equal (JE) - Checking User Input

This takes a single keystroke from the user. If the user types 'Y', it prints '1'. If they type anything else, it just exits.

```

.model small
.stack 100h
.code
main proc
    mov ah, 1      ; Input function
    int 21h        ; User input is stored in AL

    cmp al, 'Y'    ; Check if input is exactly 'Y'
    je YES_TYPED
    jmp EXIT        ; Skip if not 'Y'

YES_TYPED:
    mov dl, '1'
    mov ah, 2
    int 21h

EXIT:
    mov ah, 4ch
    int 21h
main endp
end main

```

4. Jump if Not Equal (JNE) - Password/Key Check

This compares two values. If they are *different*, it triggers the jump. Here, it prints 'N' because 5 does not equal 9.

```

.model small
.stack 100h
.code
main proc
    mov al, 5
    mov bl, 9

    cmp al, bl     ; Compare AL and BL
    jne NOT_MATCH ; Jump if AL != BL
    jmp EXIT

NOT_MATCH:
    mov dl, 'N'
    mov ah, 2
    int 21h

EXIT:
    mov ah, 4ch
    int 21h
main endp
end main

```

5. Jump if Not Equal (JNE) - Building a Custom Loop

You don't always have to use the LOOP instruction. You can build your own loops using CMP and JNE. This prints a character 3 times.

```
.model small
.stack 100h
.code
main proc
    mov cl, 3      ; Set our custom counter to 3

MY_LOOP:
    mov dl, '*'    ; Print a star
    mov ah, 2
    int 21h

    dec cl        ; Decrease counter by 1
    cmp cl, 0      ; Check if counter has hit 0
    jne MY_LOOP    ; If counter is NOT 0, jump back up

    mov ah, 4ch
    int 21h
main endp
end main
```

6. Jump if Carry (JC) - Detecting Addition Overflow

An 8-bit register (like AL) can only hold up to 255. If you add to it and it exceeds 255, the Carry Flag turns on. JC catches this.

```
.model small
.stack 100h
.code
main proc
    mov al, 255    ; AL is maxed out
    add al, 5      ; Adding 5 causes an overflow (carry)

    jc OVERFLOW_HAPPENED ; Jumps if Carry Flag is 1
    jmp EXIT

OVERFLOW_HAPPENED:
    mov dl, 'C'    ; Print 'C' for Carry
    mov ah, 2
    int 21h

EXIT:
    mov ah, 4ch
    int 21h
main endp
end main
```

7. Jump if Carry (JC) - Detecting Subtraction Underflow

The carry flag also triggers if you try to subtract a larger number from a smaller number (requiring a "borrow").

```
.model small
.stack 100h
.code
main proc
    mov al, 2
    sub al, 5      ; 2 - 5 requires a borrow, setting the Carry flag

    jc BORROW_NEEDED
    jmp EXIT

BORROW_NEEDED:
    mov dl, 'B'    ; Print 'B' for Borrow
```

```

    mov ah, 2
    int 21h

EXIT:
    mov ah, 4ch
    int 21h
main endp
end main

```

8. Combining JE and JNE - Basic Menu Choice

This checks a user's input. If they press '1', it prints 'A'. If they press '2', it prints 'B'.

```

.model small
.stack 100h
.code
main proc
    mov ah, 1      ; Get user input
    int 21h

    cmp al, '1'
    je CHOICE_1

    cmp al, '2'
    je CHOICE_2

    jmp EXIT      ; If neither, just exit

CHOICE_1:
    mov dl, 'A'
    jmp PRINT

CHOICE_2:
    mov dl, 'B'

PRINT:
    mov ah, 2
    int 21h

EXIT:
    mov ah, 4ch
    int 21h
main endp
end main

```

9. Finding the Maximum of Two Numbers

This is a classic lab question. It compares AL and BL, figures out which one is bigger using conditional jumps, and prints it.

```

.model small
.stack 100h
.code
main proc
    mov al, '3'    ; First number (in ASCII)
    mov bl, '7'    ; Second number (in ASCII)

    cmp al, bl
    jc BL_IS_BIGGER ; If AL < BL, subtracting them would cause a carry.

    ; If we didn't jump, AL is bigger or equal
    mov dl, al
    jmp PRINT_MAX

BL_IS_BIGGER:

```

```

    mov dl, bl

PRINT_MAX:
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

10. The IF-ELSE Structure in Assembly

High-level languages use if-else. In assembly, you build this using CMP, JNE (to skip the "if" part), and JMP (to skip the "else" part).

```

.model small
.stack 100h
.code
main proc
    mov al, 4
    mov bl, 4

    ; IF AL == BL
    cmp al, bl
    jne ELSE_BLOCK ; If they don't match, jump to the ELSE section

    ; --- IF BLOCK ---
    mov dl, 'T' ; True
    jmp END_IF ; Skip the ELSE block

ELSE_BLOCK:
    ; --- ELSE BLOCK ---
    mov dl, 'F' ; False

END_IF:
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main

```

User prompt: Loop related 10 code

Response: Here are 10 completely distinct, runnable examples focusing entirely on loops and nested loops. These cover everything from basic repetition to building shapes, which are incredibly common lab tasks.

1. Basic LOOP - Printing the Same Character

The easiest way to use a loop. CX acts as the counter. This prints the letter 'A' 5 times.

```

.model small
.stack 100h
.code
main proc
    mov cx, 5 ; Set loop counter to 5
    mov dl, 'A' ; The character to print

PRINT_A:
    mov ah, 2

```

```

    int 21h        ; Print 'A'

    loop PRINT_A   ; Decrement CX. If CX is not 0, jump to PRINT_A

    mov ah, 4ch
    int 21h
main endp
end main

```

2. Ascending Sequence (1 to 5)

By changing the character inside the loop, we can print a sequence.

```

.model small
.stack 100h
.code
main proc
    mov cx, 5      ; Loop 5 times
    mov dl, '1'    ; Start with ASCII '1'

ASCENDING:
    mov ah, 2
    int 21h        ; Print current number

    inc dl         ; Increment DL ('1' becomes '2', etc.)

    loop ASCENDING

    mov ah, 4ch
    int 21h
main endp
end main

```

3. Descending Sequence (5 down to 1)

Instead of incrementing, we start at a high number and decrement it each loop.

```

.model small
.stack 100h
.code
main proc
    mov cx, 5      ; Loop 5 times
    mov dl, '5'    ; Start with ASCII '5'

DESCENDING:
    mov ah, 2
    int 21h        ; Print current number

    dec dl         ; Decrement DL ('5' becomes '4', etc.)

    loop DESCENDING

    mov ah, 4ch
    int 21h
main endp
end main

```

4. Printing the Alphabet (A to Z)

This is a classic lab exam question. We loop 26 times, incrementing the letter.

```

.model small
.stack 100h
.code

```

```

main proc
    mov cx, 26      ; 26 letters in the alphabet
    mov dl, 'A'     ; Start at 'A'

ALPHABET:
    mov ah, 2
    int 21h        ; Print current letter

    inc dl         ; Move to the next letter

    loop ALPHABET

    mov ah, 4ch
    int 21h
main endp
end main

```

5. Custom Loop without the LOOP Instruction

Sometimes you want to use a register other than CX for your counter. Here is how you build a loop using CMP and JNE.

```

.model small
.stack 100h
.code
main proc
    mov bl, 0       ; Use BL as our custom counter, starting at 0
    mov dl, '*'

CUSTOM_LOOP:
    mov ah, 2
    int 21h        ; Print star

    inc bl         ; Increase our counter
    cmp bl, 4      ; Check if we have printed 4 stars
    jne CUSTOM_LOOP ; If NOT equal to 4, jump back up

    mov ah, 4ch
    int 21h
main endp
end main

```

6. Loop to Calculate a Sum (1+2+3)

Loops aren't just for printing! This calculates 1+2+3 and prints the result (6).

```

.model small
.stack 100h
.code
main proc
    mov cx, 3       ; We want to add 3 numbers
    mov bl, 0       ; BL will hold our running total
    mov al, 1       ; AL is the number we are adding (starts at 1)

SUM_LOOP:
    add bl, al      ; Add AL to total (BL)
    inc al          ; Next time, add the next number
    loop SUM_LOOP

    ; Convert total to ASCII and print
    mov dl, bl
    add dl, 48
    mov ah, 2
    int 21h

```

```

    mov ah, 4ch
    int 21h
main endp
end main

```

7. Infinite Loop (Reading until Enter is pressed)

Not all loops know how many times they will run. This reads user input repeatedly and only stops if the user presses the 'Enter' key (ASCII 13).

```

.model small
.stack 100h
.code
main proc
READ_INPUT:
    mov ah, 1      ; Input function
    int 21h

    cmp al, 13     ; 13 is the ASCII code for Enter (Carriage Return)
    je END_LOOP   ; If it is Enter, break the loop

    jmp READ_INPUT ; Otherwise, go back and read another character

END_LOOP:
    mov ah, 4ch
    int 21h
main endp
end main

```

8. Nested Loop - Printing a 2x4 Rectangle Block

When putting a loop inside a loop, both want to use CX. You **must** save the outer CX to the stack (PUSH CX) and retrieve it (POP CX) so they don't interfere.

```

.model small
.stack 100h
.code
main proc
    mov cx, 2      ; Outer loop: 2 rows

ROW_LOOP:
    push cx        ; SAVE outer loop counter

    mov cx, 4      ; Inner loop: 4 columns
COL_LOOP:
    mov dl, '#'    ; Print a hashtag
    mov ah, 2
    int 21h
    loop COL_LOOP ; Decrements the inner CX

    ; Print Newline (Line feed + Carriage return)
    mov dl, 10
    mov ah, 2
    int 21h
    mov dl, 13
    int 21h

    pop cx         ; RESTORE outer loop counter
    loop ROW_LOOP ; Decrements the outer CX

    mov ah, 4ch
    int 21h
main endp
end main

```

9. Nested Loop - Number Pattern (111, 222, 333)

This nested loop changes the character it prints based on which row it is on.

```
.model small
.stack 100h
.code
main proc
    mov cx, 3      ; 3 Rows
    mov bl, '1'    ; Start printing '1's

OUTER:
    push cx
    mov cx, 3      ; 3 numbers per row

INNER:
    mov dl, bl     ; Move current number to DL for printing
    mov ah, 2
    int 21h
    loop INNER

    ; Print Newline
    mov dl, 10
    mov ah, 2
    int 21h
    mov dl, 13
    int 21h

    inc bl         ; Increment the number for the next row (1->2)

    pop cx
    loop OUTER

    mov ah, 4ch
    int 21h
main endp
end main
```

10. Nested Loop - Right-Angled Triangle

This is the ultimate loop test. The inner loop needs to run a different number of times for each row (1 time on row 1, 2 times on row 2, etc.).

```
.model small
.stack 100h
.code
main proc
    mov bx, 1      ; BX will track which row we are on (1 to 4)

TRIANGLE_ROW:
    cmp bx, 5      ; If we reach row 5, we are done
    je END_PROG

    mov cx, bx     ; Set inner loop counter EQUAL to the current row number

TRIANGLE_COL:
    mov dl, '*'
    mov ah, 2
    int 21h
    loop TRIANGLE_COL

    ; Print Newline
    mov dl, 10
    mov ah, 2
```



```

    int 21h
    mov dl, 13
    int 21h

    inc bx          ; Move to the next row
    jmp TRIANGLE_ROW ; Jump back up

END_PROG:
    mov ah, 4ch
    int 21h
main endp
end main

```

User prompt: Explain how to write these code? I want to understand how to write this, like input, output, loop, how they implement and how they work everything regarding assembly language

Response: Moving from just reading Assembly to actually writing it requires a complete shift in how you think about programming. In high-level languages like Python or C, you tell the computer *what* to do. In Assembly, you have to tell the CPU exactly *how* to do it, step by step.

To write this code, you need to understand three core concepts: **Registers, Interrupts, and the ASCII trap.**

Here is the breakdown of how to think like a microprocessor.

1. Registers: The CPU's Pockets

The CPU doesn't have variables like `x = 5`. Instead, it has a few built-in storage slots called **Registers**. In 8086 Assembly (16-bit), you have four main general-purpose registers:

- **AX (Accumulator):** Used for math (Multiply and Divide *force* you to use AX).
- **BX (Base):** Used for storing data and memory addresses.
- **CX (Counter):** The CPU automatically looks at CX when you use a LOOP.
- **DX (Data):** Used specifically for Input and Output.

The Half-Pockets: Each 16-bit register is split into two 8-bit halves: High (H) and Low (L).

- AX is made of AH and AL.
- If you just need to store a single character (like 'A' or '5'), you only need an 8-bit half, like AL or DL.

2. The Command: MOV

The most common instruction you will write is MOV. It means "copy".

- `mov ax, 5` means "put the number 5 into AX."
- `mov bl, al` means "copy whatever is in AL into BL."

3. Interrupts (INT 21h): Talking to DOS

Your program can't directly read the keyboard or draw on the screen. It has to ask the Operating System (DOS) to do it. You do this using `int 21h` (Interrupt 21 hex).

Think of `int 21h` like a waiter at a restaurant. To place an order, you have to tell the waiter what you want by putting a specific menu number into the **AH register**.

How to write an INPUT:

You want the user to type a single key.

1. Put 1 in AH (Menu item 1: Read keyboard).

2. Call the waiter (int 21h).

3. **The Rule:** The waiter will ALWAYS hand you the result in the **AL register**.

```
mov ah, 1      ; "Waiter, I want a keyboard input"
int 21h        ; (Wait for user to press a key)
; Whatever the user pressed is now sitting in AL.
mov bl, al     ; Save it to BL so we don't lose it!
```

How to write an OUTPUT:

You want to print a character to the screen.

1. Put 2 in AH (Menu item 2: Write to screen).

2. Put the character you want to print in the **DL register** (The waiter only checks DL for output).

3. Call the waiter (int 21h).

```
mov ah, 2      ; "Waiter, I want to print something"
mov dl, 'X'    ; "Here is what I want to print: X"
int 21h        ; (X appears on the screen)
```

4. The ASCII Trap (Why we Add and Subtract 48)

This trips up almost everyone. The keyboard doesn't understand numbers; it only understands characters.

If you press the 5 key, the keyboard does NOT send the number 5. It sends the ASCII code for the *character* '5', which is actually the number **53**.

- **When taking input for math:** You have to subtract 48 to turn the character '5' (53) into the real number 5. -> `sub al, 48`
- **When printing the result of math:** You have to add 48 to turn your math answer (e.g., 5) back into the printable character '5' (53). -> `add dl, 48`

5. How to write a LOOP

Writing a loop in Assembly requires three specific pieces. You don't write `for(i=0; i<5; i++)`. Instead:

1. Put your total count in CX.
2. Create a label (a name with a colon, like `MY_LOOP:`).
3. Write `loop MY_LOOP` at the bottom.

Every time the CPU hits the word `loop`, it secretly does two things: It subtracts 1 from CX, and if CX is not zero, it jumps back to your label.

```
mov cx, 3      ; 1. Set the counter to 3

PRINT_STAR:    ; 2. Create a label to jump back to
mov ah, 2      ; (Setup output)
mov dl, '*'    ; (Load star)
int 21h        ; (Print star)

loop PRINT_STAR ; 3. CX=CX-1. If CX != 0, go back to PRINT_STAR.
```

The Standard Template

Every program you write for this lab will follow this exact skeleton. You just fill in the middle.

```
.model small    ; Tells the assembler to use a small memory model
.stack 100h     ; Reserves memory for background operations

.code          ; Your instructions start here
main proc
```

```

; =====
; YOUR LOGIC GOES HERE
; (Input, Math, Loops, Output)
; =====

mov ah, 4ch    ; "Waiter, Menu item 4Ch: Terminate Program"
int 21h        ; Execute the termination
main endp
end main

```

User prompt: How do I declare and use variables in the .data segment instead of just using registers?

Response: Using variables in the .data segment makes your assembly programs much easier to read and maintain. Instead of trying to keep track of which number is sitting in BL versus CL, you can store them in memory with meaningful names.

Here is a step-by-step guide on how to declare, initialize, and use variables in 8086 Assembly.

1. Declaring Variables in .data

Variables are placed inside the .data block before .code. You define them by specifying their **Name**, **Size**, and **Initial Value**.

Common Data Sizes:

- **DB (Define Byte):** Allocates **1 byte** (8 bits). Use this for individual ASCII characters, small numbers (0 to 255), or strings.
- **DW (Define Word):** Allocates **2 bytes** (16 bits). Use this for larger numbers (0 to 65,535).

Syntax Examples:

```

.data
num1 DB 5          ; 8-bit variable initialized to 5
num2 DW 1000        ; 16-bit variable initialized to 1000
user_input DB ?     ; '?' means uninitialized (reserved space)
msg DB 'Hello!$'    ; String array (MUST end with '$' for DOS string printing)

```

2. The Golden Rule: Initializing the Data Segment (DS)

When your program starts, the CPU does **not** automatically know where your .data segment is located in memory. You must manually link the .data segment address to the DS (**Data Segment**) register at the start of main proc.

 **If you forget this step, your program will print garbage or crash!**

```

main proc
mov ax, @data    ; Get the memory address of the .data segment
mov ds, ax       ; Move that address into the Data Segment register

; Now you can safely use your variables!

```

3. How to Use Variables in Code

A. Moving Data Between Variables and Registers

- **Read from memory:** `mov al, num1` (*Copies the value of num1 into AL*)
- **Write to memory:** `mov user_input, al` (*Copies what is in AL into user_input*)

 **The Assembly Restriction:** You **cannot** move memory directly to memory!

Writing `mov num2, num1` is illegal. You must route it through a register first:

```
mov al, num1
mov num2, al
```

B. Size Matching

The register size must match the variable size:

- `mov al, num1` ✓ (Both are 8-bit / Byte)
- `mov ax, num2` ✓ (Both are 16-bit / Word)
- `mov ax, num1` ✗ (Error: Size mismatch! 16-bit register with 8-bit variable)

4. Printing Strings with Function 9h

To print an entire string instead of single characters, use DOS **Function 9**:

1. Load 9 into AH.
2. Load the address of the string into DX using LEA (Load Effective Address).
3. Ensure the string in `.data` ends with a `$`.

```
mov ah, 9
lea dx, msg      ; LEA loads the memory address of 'msg' into DX
int 21h
```

5. Complete Runnable Example

Here is a full program that uses strings, saves user input into a variable, and reads it back to display it:

```
.model small
.stack 100h

.data
    prompt DB 'Enter a single digit: $'
    result DB 10, 13, 'You stored: $' ; 10, 13 creates a new line
    saved_num DB ? ; Variable to hold user input

.code
main proc
    ; Step 1: Initialize Data Segment
    mov ax, @data
    mov ds, ax

    ; Step 2: Print the prompt string
    mov ah, 9
    lea dx, prompt
    int 21h

    ; Step 3: Get user input
    mov ah, 1
    int 21h ; Input stored in AL

    ; Step 4: Save input into our variable
    mov saved_num, al

    ; Step 5: Print the result string
    mov ah, 9
    lea dx, result
    int 21h

    ; Step 6: Load the variable back into DL and print it
```

```

    mov ah, 2
    mov dl, saved_num          ; Read from variable
    int 21h

    ; Step 7: Terminate program
    mov ah, 4ch
    int 21h
main endp
end main

```

User prompt: More concept and code about assembly language

Response: To take your assembly skills to the next level, you need to master how the CPU handles **Memory Arrays**, **Bit Manipulation**, the **Stack**, and **Functions (Procedures)**.

These four core concepts form the backbone of advanced 8086 assembly programming and appear frequently in lab exams.

1. Arrays & Index Registers (SI / DI)

In high-level languages, you access arrays using `arr[i]`. In Assembly, an array is just a sequence of bytes stored side-by-side in memory.

To traverse an array, we use **Index Registers**:

- **SI (Source Index)**: Holds the memory address offset of the current array element.
- **INC SI**: Steps to the next byte in the array.

Code: Printing Elements of an Array

```

.model small
.stack 100h

.data
    arr DB 'H', 'e', 'l', 'l', 'o' ; Array of 5 bytes
    len EQU 5                      ; EQU defines a constant length

.code
main proc
    mov ax, @data
    mov ds, ax

    lea si, arr                    ; Load the starting memory address of 'arr' into SI
    mov cx, len                   ; Set loop counter to array length

PRINT_ARRAY:
    mov dl, [si]                  ; Square brackets [] mean "get data from memory address stored in SI"
    mov ah, 2
    int 21h                       ; Print character

    inc si                        ; Move SI to point to the next array element
    loop PRINT_ARRAY

    mov ah, 4ch
    int 21h
main endp
end main

```

2. The Stack (PUSH & POP)

The Stack is a **LIFO (Last-In, First-Out)** memory area. It works like a stack of plates: the last plate you put on top is the first one you take off.

- **PUSH reg:** Saves a 16-bit register value onto the top of the stack.
- **POP reg:** Removes the top value from the stack and stores it into a register.

 **Rule:** The stack operates strictly in **16-bit (Word)** chunks. You can PUSH AX or PUSH BX, but you cannot push 8-bit registers like AL or BL.

Code: Reversing a String Using the Stack

```
.model small
.stack 100h

.data
    str DB 'CAT$'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; Step 1: Push characters onto the Stack ('C', then 'A', then 'T')
    lea si, str

    mov ax, 0
    mov al, [si]      ; Load 'C'
    push ax           ; PUSH 16-bit AX

    mov al, [si+1]    ; Load 'A'
    push ax

    mov al, [si+2]    ; Load 'T'
    push ax

    ; Step 2: Pop them off (They come out in reverse: 'T', then 'A', then 'C')
    mov cx, 3
PRINT_REVERSE:
    pop ax             ; POP top value into AX
    mov dl, al         ; DL gets the character
    mov ah, 2
    int 21h            ; Print character
    loop PRINT_REVERSE

    mov ah, 4ch
    int 21h
main endp
end main
```

3. Logical Operations & Bit Manipulation

Bitwise logic allows you to manipulate raw bits inside registers.

Instruction	Operation	Common Use Case
AND	Both bits must be 1	Masking out unwanted bits, checking Odd/Even
OR	At least one bit is 1	Forcing specific bits to turn ON (1)
XOR	Bits must be different	Clearing registers (XOR AX, AX is faster than MOV AX, 0)

Instruction	Operation	Common Use Case
TEST	Performs AND without saving result	Non-destructive bit checking

Code: Checking if a Number is Odd or Even

The last bit (Bit 0) of any odd binary number is always 1. The last bit of an even number is always 0.

```
.model small
.stack 100h

.code
main proc
    mov al, 7                ; Binary: 0000 0111

    test al, 1              ; Performs (AL AND 1). Sets Zero Flag (ZF=1) if result is 0
    jz IS_EVEN              ; Jump if Zero Flag is set (means last bit was 0)

IS_ODD:
    mov dl, 'O'             ; 'O' for Odd
    jmp PRINT

IS_EVEN:
    mov dl, 'E'             ; 'E' for Even

PRINT:
    mov ah, 2
    int 21h

    mov ah, 4ch
    int 21h
main endp
end main
```

4. Procedures (PROC, CALL, RET)

To avoid repeating the same code (like printing a newline) multiple times, you write **Procedures** (Assembly functions).

- **CALL proc_name:** Jumps to the procedure and pushes the return address onto the stack.
- **RET:** Pops the return address and jumps back to where CALL was made.

Code: Reusable Newline Procedure

```
.model small
.stack 100h

.code
main proc
    ; Print 'A'
    mov dl, 'A'
    mov ah, 2
    int 21h

    call NEWLINE           ; Call our procedure

    ; Print 'B'
    mov dl, 'B'
    mov ah, 2
    int 21h

    mov ah, 4ch
```

```

    int 21h
main endp

; =====
; PROCEDURE: Prints Carriage Return + Line Feed
; =====
NEWLINE proc
    mov ah, 2
    mov dl, 10      ; Line feed
    int 21h
    mov dl, 13      ; Carriage return
    int 21h
    ret             ; Return back to main
NEWLINE endp

end main

```

Would you like to explore array algorithms (such as finding the maximum element or searching), or would you prefer practicing how to handle multi-digit number input and output?

User prompt: No, give more concept

Response: Here are four essential advanced topics in 8086 Assembly Language that will give you complete mastery over microprocessor programming: **Shift & Rotate Operations**, **String Instructions**, **Macros**, and **Multi-Digit Number Output**.

1. Shift & Rotate Instructions (Bit Shifting)

Bit shifts are used for fast multiplication/division by powers of 2 (2, 4, 8, ...) and for inspecting individual bits inside a byte.

- **SHL (Shift Left):** Shifts all bits to the left. Fills the rightmost bit with 0. The bit pushed off on the left goes into the Carry Flag (CF).
 - *Effect: Multiplies by 2* per shift.
- **SHR (Shift Right):** Shifts all bits to the right. Fills the leftmost bit with 0.
 - *Effect: Divides by 2* per shift.
- **ROL (Rotate Left):** Bits shift left, and the bit pushed off the left **wraps around** to fill the rightmost position.
- **ROR (Rotate Right):** Bits shift right and wrap around to the left.

```

.model small
.stack 100h
.code
main proc
    mov al, 4      ; Binary: 0000 0100 (Decimal 4)

    shl al, 1      ; Shift left 1 bit -> Binary: 0000 1000 (Decimal 8)
    shl al, 1      ; Shift left 1 bit -> Binary: 0001 0000 (Decimal 16)

    ; AL now holds 16! (Fast multiplication: 4 * 2 * 2 = 16)

    mov al, 20     ; Decimal 20
    shr al, 1      ; Shift right 1 bit -> AL becomes 10 (20 / 2)

    mov ah, 4ch
    int 21h
main endp
end main

```

2. String & Block Memory Instructions

Instead of manually writing loops with **SI** to process strings or memory blocks, 8086 Assembly has built-in string hardware instructions.

When using these, **SI** points to the source address (**DS:SI**), **DI** points to the destination address (**ES:DI**), and **CX** holds the count.

Instruction	Action
CLD	Clears Direction Flag (processes memory forward : SI & DI auto-increment).
STD	Sets Direction Flag (processes memory backward : SI & DI auto-decrement).
LODSB	Loads byte from [SI] into AL , then auto-increments SI .
STOSB	Stores byte from AL into [DI], then auto-increments DI .
MOVSB	Copies byte from [SI] directly to [DI], auto-incrementing both.
REP	Repeat prefix! Executes the string instruction CX times automatically.

Code: Fast Memory Copy using **REP MOVSB**

This copies 5 bytes from **src** to **dest** in hardware without an explicit assembly loop instruction:

```
.model small
.stack 100h

.data
    src DB 'HELLO'
    dest DB 5 DUP(?) ; Reserve 5 uninitialized bytes

.code
main proc
    mov ax, @data
    mov ds, ax
    mov es, ax ; ES must also point to Data Segment for destination operations

    cld ; Set direction to FORWARD
    lea si, src ; SI = Source
    lea di, dest ; DI = Destination
    mov cx, 5 ; Repeat 5 times

    rep movsb ; Hardware repeats: Copy byte from [SI] to [DI], increment SI & DI, decrement CX

    mov ah, 4ch
    int 21h
main endp
end main
```

3. Macros vs. Procedures

While **Procedures** (**CALL/RET**) save memory by jumping to reusable code blocks, they introduce a small CPU time penalty because of stack operations.

A **Macro** is expanded *inline* by the assembler before compilation. It replaces the macro name with the actual raw instructions every time it is used.

- **Use Macros for:** Short, repetitive tasks (like printing a character or newline).
- **Use Procedures for:** Large, complex algorithms.

```
; =====
; MACRO DEFINITION (Must be placed BEFORE code)
```

```
; =====
PRINT_CHAR MACRO char
    mov ah, 2
    mov dl, char
    int 21h
ENDM

.model small
.stack 100h
.code
main proc
    ; Using the Macro:
    PRINT_CHAR 'X'    ; Expands inline to mov ah,2 / mov dl,'X' / int 21h
    PRINT_CHAR 'Y'    ; Expands inline again
    PRINT_CHAR 'Z'

    mov ah, 4ch
    int 21h
main endp
end main
```

4. Multi-Digit Number Printing Algorithm

So far, all single-digit math outputted numbers from 0 to 9 by adding 48 (add dl, 48). But what if your addition or multiplication result is 125?

If you try to print 125 directly, DOS will try to print the ASCII character #125 (}).

The Algorithm to Print Any Multi-Digit Number:

1. Divide the number by **10**.
2. Save the **Remainder** (which is the last digit) onto the **Stack**.
3. Repeat division with the **Quotient** until the quotient becomes 0.
4. **Pop** digits off the Stack one-by-one and print them (they will come off in left-to-right order).

```
.model small
.stack 100h
.code
main proc
    mov ax, 125    ; The multi-digit number we want to print
    mov cx, 0      ; Counter to track how many digits we push
    mov bx, 10     ; Divisor

CONVERT_LOOP:
    mov dx, 0      ; Clear DX before 16-bit division (DX:AX / BX)
    div bx         ; AX = Quotient, DX = Remainder

    push dx        ; Push the remainder (digit) to stack
    inc cx         ; Increment digit count

    cmp ax, 0      ; Is quotient 0?
    jne CONVERT_LOOP ; If not 0, keep dividing

PRINT_LOOP:
    pop dx         ; Pop the last digit pushed
    add dl, 48     ; Convert to ASCII
    mov ah, 2
    int 21h        ; Print digit
    loop PRINT_LOOP ; Loop until all digits are printed

    mov ah, 4ch
```

```
int 21h
main endp
end main
```

Summary of the Flags Register

When you run `CMP` or math operations, the CPU silently sets or clears bits in the **Flags Register**. Conditional jumps rely entirely on these flags:

Flag Name	Symbol	Set to 1 When...
Zero Flag	ZF	The result of an operation is zero (e.g., $5 - 5 = 0$).
Carry Flag	CF	An operation causes an unsigned overflow/borrow.
Sign Flag	SF	The result of an operation is negative (MSB is 1).
Overflow Flag	OF	A signed arithmetic operation exceeds limits.

 **Did you know?** The `CMP AL, BL` instruction is secretly just a `SUB AL, BL` subtraction! The CPU calculates `AL - BL`, sets the Flags (ZF, CF, SF), and then throws away the subtraction result without altering AL.

Would you like to walk through a complete, full-scale lab program (such as an interactive calculator or array sorting algorithm) that combines all these concepts?
